

JC4004 – Computational Intelligence

Sessions 15 & 16: Artificial neural networks

Dr Jari Korhonen (jari.korhonen@abdn.ac.uk)

Dr Yongchao Huang (Yongchao.huang@abdn.ac.uk)

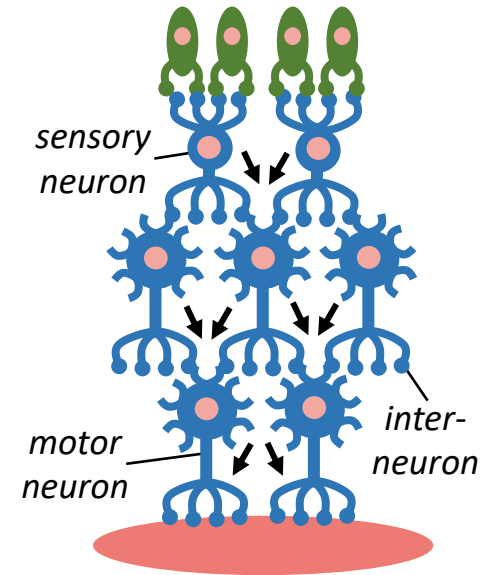
Nov-Dec 2024

What are artificial neural networks?

- *Artificial neural networks (ANN)* or simply *neural networks* form the basis for most of the modern AI system
 - Consists of connected units called *artificial neurons* or *perceptrons*
 - Inspiration from biological neural networks in animal brains
- Artificial neural networks already introduced in other courses
 - Some content in the following sessions will be redundant with other courses; however, some repetition cannot be avoided, as we aim at self-contained course content

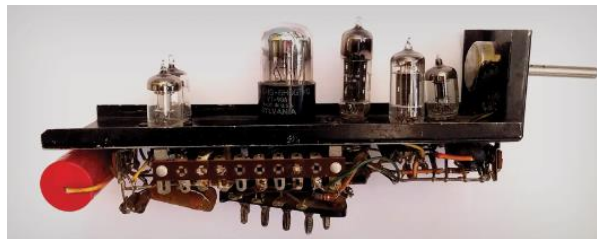
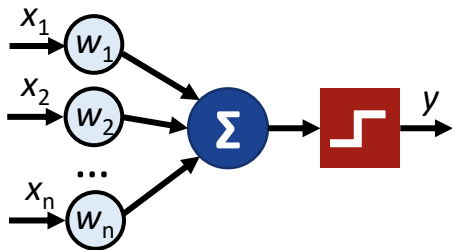
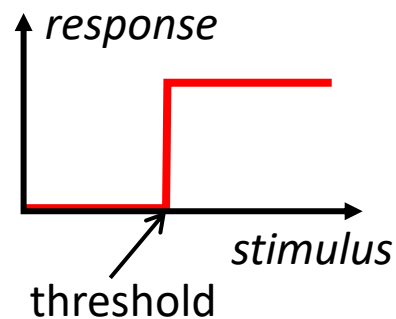
Biological neurons

- There are three basic types of biological neurons:
 - *Sensory neurons* are connected to sensory organs such as eye or ear, and they respond to stimuli like sound or light
 - *Motor neurons* control e.g., muscle contractions based on their input signal
 - *Interneurons* connect neurons to other neurons to form the *nervous system*
- Neurons communicate with each other via electrical and chemical signaling



From biological to artificial neurons

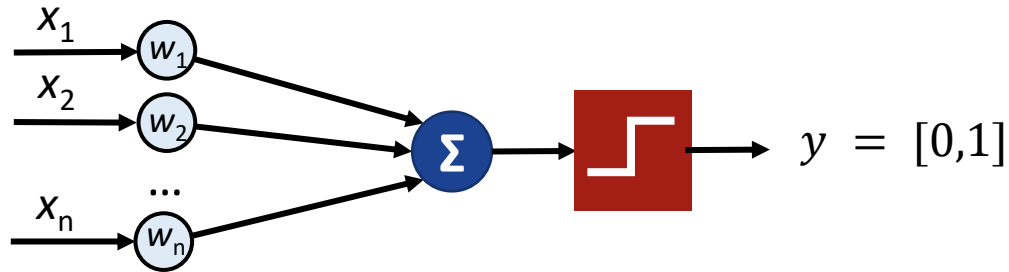
- Biological neurons produce output signal (*response*) only if the input signal (*stimulus*) exceeds a threshold
 - Nerve impulses follow *all-or-none law*: a neuron gives either a full response, or no response at all
- Artificial neurons loosely simulate biological neurons
 - First artificial neurons were hardware implementations



The classic perceptron

- Perceptron is a type of artificial model of a biological neuron introduced first by *Mark Rosenblatt* in 1957:
 - Output y computed from input $\mathbf{x} = \{x_1, \dots, x_n\}$ using weights $\mathbf{w} = \{w_1, \dots, w_n\}$ and threshold θ :

$$y = 1, \text{ if } \sum_{i=1}^n w_i x_i \geq \theta, \text{ else } y = 0$$

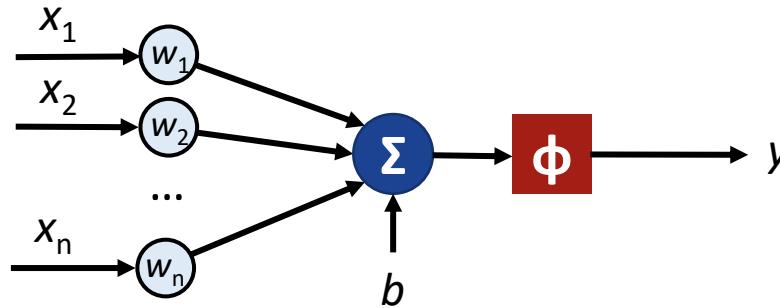


The modern perceptron

- In modern neural networks, usually the version of perceptron with bias b and activation function φ is used:

$$y = \varphi \left(b + \sum_{i=1}^n w_i x_i \right)$$

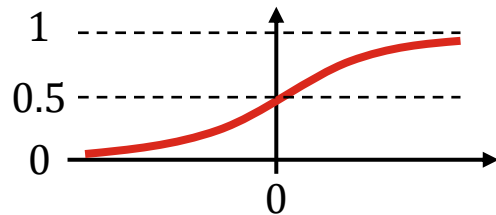
- Note that we can choose different activation functions φ !



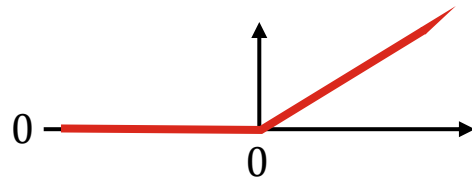
Sigmoid and ReLU activation functions

- Many activation functions have been described in research literature
- In the early neural networks, *Sigmoid function* was typically used as activation function
 - The benefit is that output is limited to $[0,1]$
- Today, *rectified linear unit* (ReLU) is the most common activation function
 - Works usually as well as Sigmoid or even better, and it is simpler to compute

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



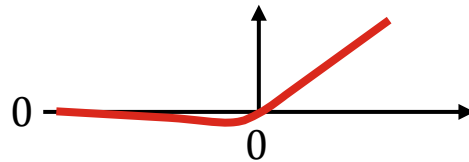
$$\text{ReLU}(x) = \max(0, x)$$



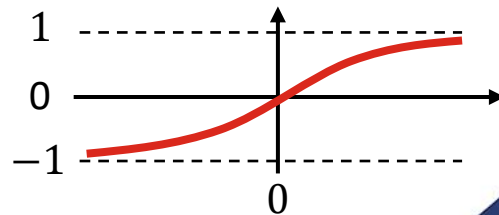
Other activation functions

- *Gaussian error linear unit* (GELU) is often used as a continuous approximation of ReLU
 - The benefit is that GELU is differentiable
- *Hyperbolic tangent* (tanh) is sometimes used as an alternative to Sigmoid
 - Range is $[-1,1]$ instead of $[0,1]$
- Also, many others, but most of them not commonly used

$$\text{GELU}(x) = \frac{1}{2}x \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right) \right)$$

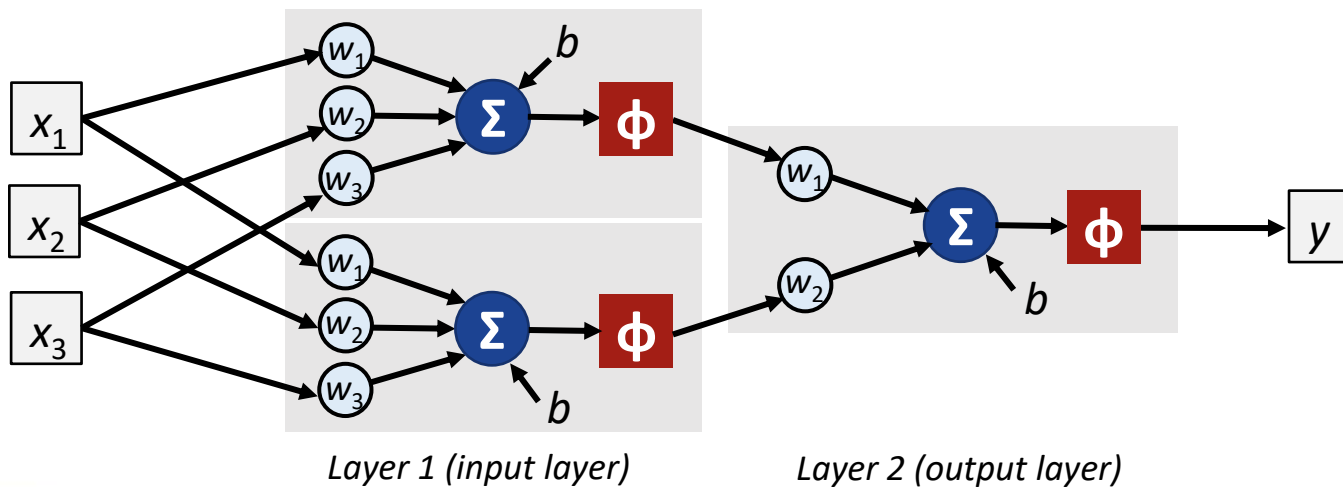


$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



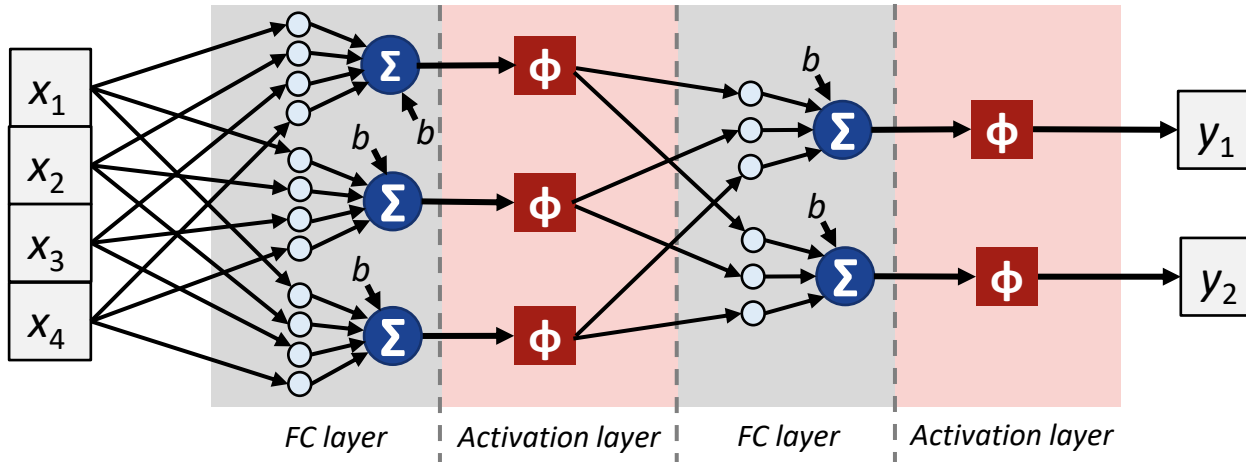
Multilayer perceptron

- *Multilayer perceptron (MLP)* is an artificial neural network where perceptrons are arranged in two or more layers
 - Outputs from the previous layer are used as inputs to the following layer



Fully connected and activation layers

- In modern neural networks, all inputs are usually connected to all the neurons; therefore, it can be called *fully connected (FC)* or *linear* layer
 - Activation functions usually represented by a separate *activation layer*



Mathematical formulations

- Fully connected layer can be expressed as:

$$\mathbf{y} = \mathbf{x}\mathbf{W} + \mathbf{b},$$

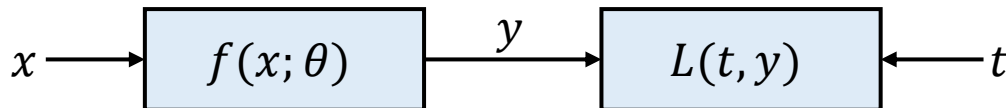
- $\mathbf{x}$ and $\mathbf{y}$ are the input and output vectors (usually of different lengths)
 - $\mathbf{W}$ is the weight matrix (each row represents an element of the input vector, and each column represents an element in the output vector)
 - $\mathbf{b}$ is the bias vector of same length as the output vector
- The whole network can be expressed using nested functions:

$$\mathbf{y} = f_n(\dots f_2(f_1(\mathbf{x})) \dots) = f(\mathbf{x})$$

- Function f_1 represents the first and f_n represents the last layer

Loss functions

- To train the network to make meaningful predictions, we need to define loss function $L(t, y)$ where t is the target and y is the output
 - Small value of L means that t and y are close to each other
 - For regression problems, we can use squared error: $L(t, y) = (t - y)^2$
 - Given $y = f(x; \theta)$, where θ are the learnable network parameters, network training aims to find θ minimising $L(t, f(x; \theta))$



Loss functions

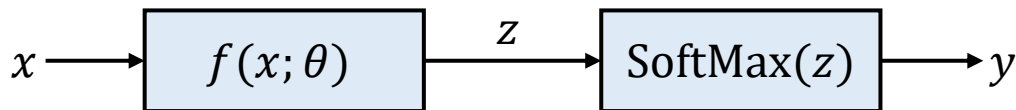
- To train the network to make meaningful predictions, we need to define loss function $L(t, y)$ where t is the target and y is the output
 - Small value of L means that t and y are close to each other
 - For regression problems, we can use squared error: $L(t, y) = (t - y)^2$
 - Given $y = f(x; \theta)$, where θ are the learnable network parameters, network training aims to find θ minimising $L(t, f(x; \theta))$

Note that the learnable parameters θ are often referred to as “weights,” even though θ is **not** limited to the weights of the linear layers only!

SoftMax

- Neural networks are often used for classification tasks
 - In classification tasks, output $y = \{y_1, \dots, y_d\}$ is a vector of length d where each element represents one of the possible classes
 - The range of values z obtained from a linear layer can be arbitrary, so the values should be normalised into a probability distribution
 - Usually, SoftMax function is used for normalisation:

$$y_i = \text{SoftMax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^d \exp(z_j)}$$



Cross-entropy loss

- Cross-entropy loss is the most common loss function for classification
 - Measures the distance between two probability distributions
 - Output y is the probability distribution predicted by the neural network, and target t is the ground truth probability distribution
- Cross-entropy loss is computed as:
$$L(t, y) = - \sum_{i=1}^d t_i \ln(y_i)$$
- Note that usually the ground truth class has target probability 1 and the other classes have target probability 0: given that the ground truth class index is c , we can simplify cross-entropy loss as: $L(c, y) = -\ln(y_c)$

SoftMax and cross-entropy example

- Let us assume that we have a neural network to predict weather categorised in three classes: 1: *sunny*, 2: *cloudy*, and 3: *rainy*
 - Given output vector $z = \{-10, 0.6, 0.5\}$ and the ground truth weather is rainy, what is the cross-entropy loss?

- First, we can normalise using SoftMax: $y_i = \frac{\exp(z_i)}{\sum_{j=1}^d \exp(z_j)}$

$$y = \frac{\exp(\{-10, 0.6, 0.5\})}{\exp(-10) + \exp(0.6) + \exp(0.5)} \approx \frac{\{4.5 \cdot 10^{-5}, 1.82, 1.65\}}{4.5 \cdot 10^{-5} + 1.82 + 1.65} \approx \{1 \cdot 10^{-5}, 0.525, 0.475\}$$

- Then, we can compute cross-entropy loss: $L = -\ln(0.475) \approx \underline{\underline{0.744}}$

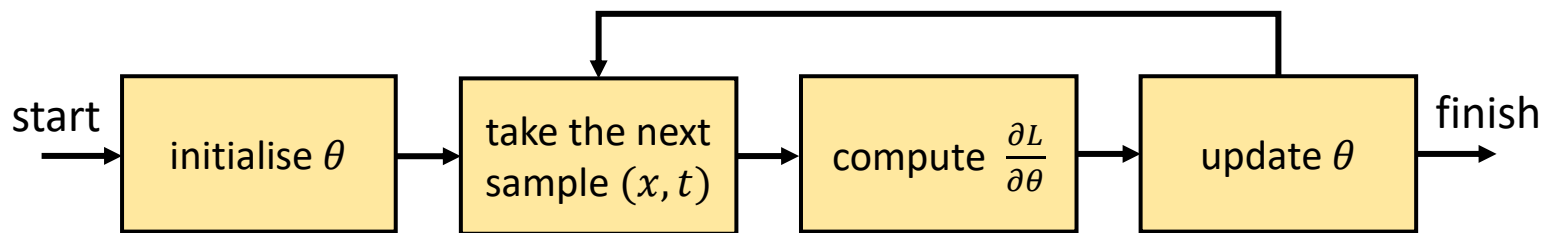
Break: any questions?

Training neural networks

- We could think of training neural networks as an optimisation problem, i.e., $\operatorname{argmin}_{\theta} L(t, f(x; \theta))$
 - However, there is no closed form solution (except in special cases), so we can only numerically approximate the optimal parameters θ
- The first deep neural networks in the 1960s and 70s were trained layer by *group method for data handling (GMDH)*
 - GMDH uses a combinatorial approach to select the best models from several candidates which does not scale well to large networks
- Today, neural networks are mostly trained using different variants of gradient-based optimisation with backpropagation

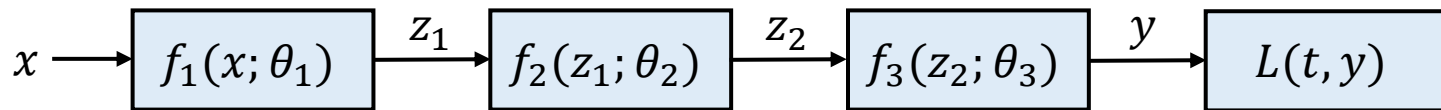
Training loop

- Network is trained iteratively by looping through the training samples of input x and target t pairs and updating the weights θ
 - First, θ is initialised (there are several methods for initialisation)
 - Gradients $\frac{\partial L}{\partial \theta}$ are computed, usually through backpropagation
 - The gradients are passed to the optimisation algorithm to update θ



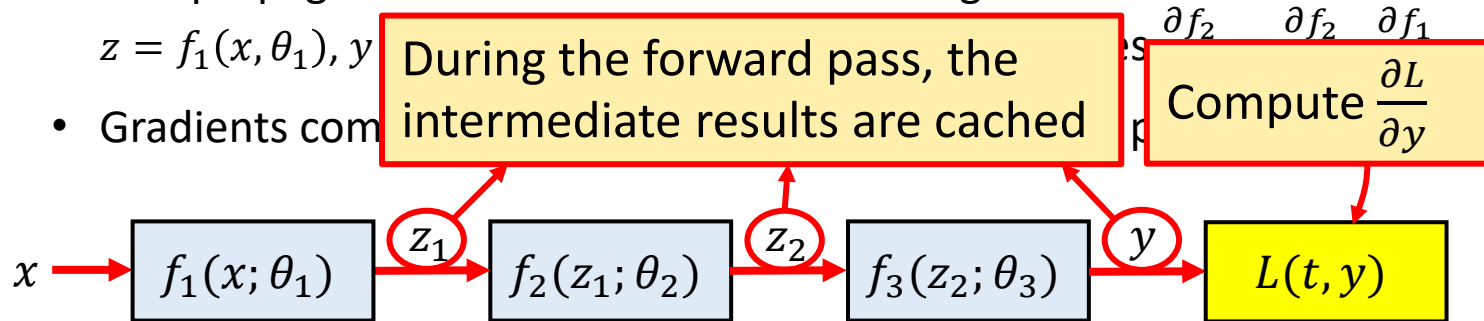
Backpropagation

- For large networks, computing the partial derivatives directly to obtain the gradients for the optimisation method can be inefficient
 - Fortunately, the layered architecture of neural networks allows the gradients to be computed through *backpropagation*
 - Backpropagation is based on the *chain rule*: given chained functions $z = f_1(x, \theta_1)$, $y = f_2(z, \theta_2)$, we can compute derivatives $\frac{\partial f_2}{\partial \theta_1} = \frac{\partial f_2}{\partial z} \cdot \frac{\partial f_1}{\partial \theta_1}$
 - Gradients computed layer by layer as the derivatives passed backwards



Backpropagation

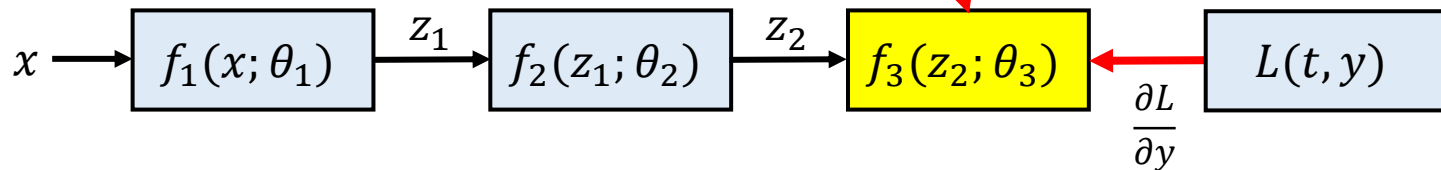
- For large networks, computing the partial derivatives directly to obtain the gradients for the optimisation method can be inefficient
 - Fortunately, the layered architecture of neural networks allows the gradients to be computed through *backpropagation*
 - Backpropagation is based on the *chain rule*: given chained functions $z = f_1(x, \theta_1), y = f_2(z_1, \theta_2), p = f_3(z_2, \theta_3)$ compute $\frac{\partial L}{\partial y}$
 - Gradients computed during the forward pass are cached



Backpropagation

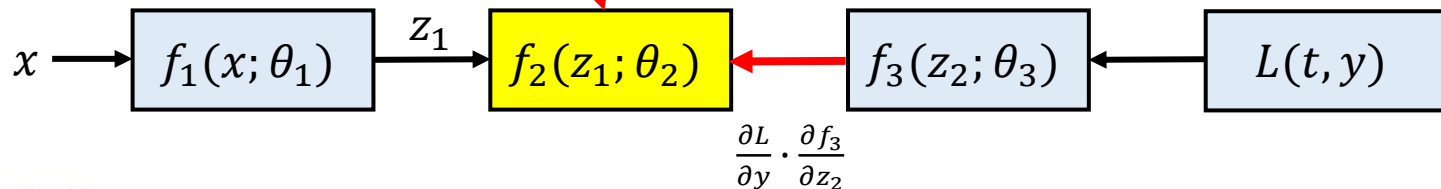
- For large networks, computing the partial derivatives directly to obtain the gradients for the optimisation method can be inefficient
 - Fortunately, the layered architecture of neural networks allows the gradients to be computed through *backpropagation*
 - Backpropagation is based on the chain rule. If $z = f_1(x, \theta_1)$, $y = f_2(z, \theta_2)$, we can compute $\frac{\partial L}{\partial \theta_3}$ by passing the gradient backwards through the layers.

Pass $\frac{\partial L}{\partial y}$ backwards, and compute $\frac{\partial L}{\partial \theta_3} = \frac{\partial L}{\partial y} \cdot \frac{\partial f_3}{\partial \theta_3}$
 - Gradients computed layer by layer and passed backwards



Backpropagation

- For large networks, computing the partial derivatives directly to obtain the gradients for the optimisation method can be inefficient
 - Fortunately, the layered architecture of neural networks allows the gradients to be computed through *backpropagation*
 - Backpropagation involves passing gradients backwards through the network
 - Pass $\frac{\partial L}{\partial y} \cdot \frac{\partial f_3}{\partial z_2}$ backwards, and compute $\frac{\partial L}{\partial \theta_2} = \frac{\partial L}{\partial y} \cdot \frac{\partial f_3}{\partial z_2} \cdot \frac{\partial f_2}{\partial \theta_2}$
 - Gradients computed at each layer are passed backwards

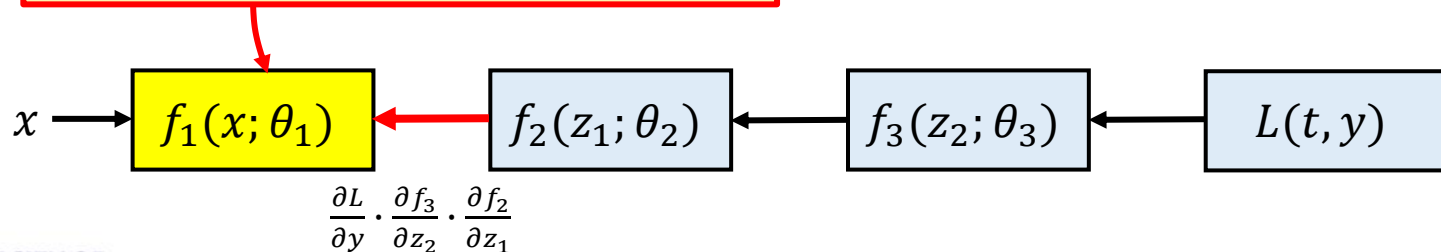


Backpropagation

- For large networks, computing the partial derivatives directly to obtain the gradients for the optimisation method can be inefficient
 - Fortunately, the layered architecture of neural networks allows the gradients to be computed through *backpropagation*

Pass $\frac{\partial L}{\partial y} \cdot \frac{\partial f_3}{\partial z_2} \cdot \frac{\partial f_2}{\partial z_1}$ backwards, and compute $\frac{\partial L}{\partial \theta_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial f_3}{\partial z_2} \cdot \frac{\partial f_2}{\partial z_1} \cdot \frac{\partial f_1}{\partial \theta_1}$ as the derivatives passed backwards

Chain rule: given chained functions compute derivatives $\frac{\partial f_2}{\partial \theta_1} = \frac{\partial f_2}{\partial z} \cdot \frac{\partial f_1}{\partial \theta_1}$



Automatic differentiation

- In software implementations of neural networks, we could write functions for computing the partial derivatives manually
 - For each function $f(x)$, we could first find the symbolic solution for the derivative $df(x)/dx$ and then implement it in the program code
- However, the prominent software libraries for machine learning provide us with tools for *automatic differentiation* (*AutoDiff*)
 - AutoDiff caches the values during the forward pass and applies the chain rule using basic operations to calculate partial derivatives
 - AutoDiff is usually more efficient than manual symbolic differentiation, and more accurate than numerical differentiation

Stochastic gradient descend

- *Stochastic gradient descend (SGD)* is the baseline optimisation method
 - Each learnable parameter θ_i (weight) updated using the gradient $\frac{\partial L}{\partial \theta_i}$:

$$\theta_i \leftarrow \theta_i - \eta \frac{\partial L}{\partial \theta_i}$$

- SGD computes gradients using the loss function computed for a single (stochastic) data point x instead of full set of training samples
- Parameter η is the learning rate defining the step size of change
- Too small value of η makes the algorithm to converge slowly, whereas too large value makes the algorithm unstable

Summary

- Most of the modern AI systems are based on *artificial neural networks*
 - The basic building block of an artificial neural network is an *artificial neuron* or *perceptron*
 - The output of a single neuron is a linear combination of the inputs multiplied by learnable weights, scaled by a nonlinear *activation function*
 - In the basic neural networks, multiple neurons are structured into layers
- Modern neural networks are usually trained using optimisation algorithms based on *stochastic gradient descend* (SGD)
 - Gradients for the individual learnable parameters (weights) computed through backpropagation and passed to the optimisation algorithm

Questions and discussion